MLOps System Architecture & Implementation Plan

Implementation Plan: Student Performance & Dropout-Risk Prediction System (Part 2 — MLOps Pipeline Extension)

1. Project Overview & Context

Part 1 established the end-to-end data engineering foundation: multi-source CSV ingestion, strict data quality validation with quarantine logs (**`data/rejected/`**), relational PostgreSQL storage (**`students`**, **`academic_performance`**), an analytical data mart (**`student_analytics`**), an Apache Airflow DAG (**`dags/student_pipeline.py`**), and a 5-view Streamlit dashboard.

Part 2 extends this existing architecture into an **enterprise-grade, reproducible MLOps platform** worth the remaining **50 Marks**. This plan details the incremental addition of:

- A leak-free academic failure risk prediction target.
- A reusable scikit-learn preprocessing and feature engineering pipeline.
- Model training, hyperparameter tuning, and comparison across **Logistic Regression**, **Random Forest**, and **XGBoost**.
- Experiment tracking, metric logging, and artifact management using **MLflow**.
- Dataset and model versioning with **DVC** and Git-based metadata.
- A production **FastAPI** prediction microservice with structured schemas, health checks, and batch prediction.
- **Docker** containerization for the inference service and integration into **`docker-compose.yml`**.
- Direct integration of real-time predictions and MLOps monitoring into the existing **Streamlit dashboard**.
- Live data quality monitoring, **Population Stability Index (PSI)** feature drift detection, and service latency tracking.
- An automated model **retraining pipeline** with candidate-versus-champion promotion criteria.
- Comprehensive automated test suite with **Pytest** and **GitHub Actions CI**.

2. Architecture & Pipeline Flow (Part 1 + Part 2)

```
flowchart TD
    subgraph Part 1: Data Engineering
        RAW["data/raw/<br/>(student-mat.csv, student-por.csv)"]
        VAL["src/validation/validate.py"]
        STG["data/staging/staging_students.csv"]
        TRF["src/transformation/transform.py"]
        MART["data/cleaned/student_analytics.csv<br/>PostgreSQL / SQLite Warehouse"]
    end

    subgraph Part 2: Feature Engineering & Versioning
        FEAT["src/features/build_features.py<br/>(ColumnTransformer, StandardScaler, OneHotEncoder)"]
        DVC_META["dvc.yaml / params.yaml<br/>data/reference/dataset_fingerprint.json"]
        REF["data/reference/reference_features.parquet"]
    end

    subgraph Part 2: Model Training & Experiment Tracking
        TRAIN["src/training/train.py & evaluate.py<br/>(Stratified Split: 70/15/15)"]
        MODELS["Logistic Regression (Baseline)<br/>Random Forest<br/>XGBoost Classifier"]
        MLFLOW["MLflow Tracking & Registry<br/>(sqlite:///mlruns.db / mlruns/)"]
        REGISTRY["models/best_model.joblib<br/>models/preprocessor.joblib<br/>models/model_registry.json"]
    end

    subgraph Part 2: Serving & Containerization
        FASTAPI["api/main.py (FastAPI Service)<br/>GET /health, GET /model-info<br/>POST /predict, POST /predict/batch<br/>GET /metrics"
        DOCKER["Docker Containerization<br/>(api/Dockerfile, docker-compose.yml)"]
    end

    subgraph Part 2: Interactive Consumption & Monitoring
        DASH["Streamlit Dashboard (dashboard/app.py)<br/>• Views 1-5 (Part 1 Analytics)<br/>• View 6: Real-Time Risk Prediction<br/>• Vi
        MON["src/monitoring/drift.py & data_quality.py<br/>(PSI Drift, Data Quality, Latency Metrics)"]
        RETRAIN["src/training/retrain.py<br/>(Champion vs Challenger Promotion Loop)"]
    end

    RAW --> VAL --> STG --> TRF --> MART
    MART --> FEAT
    FEAT --> REF
    FEAT --> DVC_META
    FEAT --> TRAIN
    TRAIN --> MODELS
    MODELS --> MLFLOW
    MLFLOW --> REGISTRY
    REGISTRY --> FASTAPI
    FASTAPI --> DOCKER
    FASTAPI --> DASH
    FASTAPI --> MON
    MON --> RETRAIN
    RETRAIN -. Conditionally Promotes .-> REGISTRY
```

3. Detailed Component Specifications

3.1 Prediction Target Definition (Zero-Leakage Academic Failure Risk)

- **Problem Formulation**: Early identification of students at high risk of academic failure before final evaluations.
- **Target Variable**: **`failure_risk`** *\in \{0, 1\}*
- **`1`** = **At-Risk / Failing**: Final grade *G3 < 10* (failing the course under Portuguese secondary school standards).
- **`0`** = **Passing**: Final grade *G3 \ge 10*.
- **Strict Leakage Prevention**:
- Exclude all post-outcome and surrogate columns: *G3*, *G2*, **`final_grade`**, **`average_grade`**, **`pass_fail`**, **`performance_category`**, **`grade_improvement`**, **`risk_status`**, and surrogate primary keys (**`student_id`**).
- Prediction is performed using only early-term / baseline information available at prediction time: Period 1 grade (*G1*), study habits, failures, absences, school engagement, demographic factors, and curriculum subject.
- **Class Balance**: 806 Passing (78.4%) vs 222 At-Risk (21.6%).
- **Evaluation Priority**: Recall on Class 1 (identifying true at-risk students) and F1-score/ROC-AUC, balanced against precision to prevent alert fatigue.

3.2 Feature Engineering Module (`src/features/build_features.py`)

- **Numerical Features**: **`g1`**, **`age`**, **`absences`**, **`attendance_rate`**, **`studytime`**, **`failures`**, **`famrel`**, **`freetime`**, **`goout`**, **`dalc`**, **`walc`**, **`health`**, **`medu`**, **`fedu`**.
- Processed using **`StandardScaler()`**.

• **Categorical Features**: **`school`**, **`sex`**, **`address`**, **`famsize`**, **`pstatus`**, **`mjob`**, **`fjob`**, **`reason`**, **`guardian`**, **`parent_education`**, **`study_time_category`**, **`subject`**, **`schoolsup`**, **`famsup`**, **`paid`**, **`activities`**, **`nursery`**, **`higher`**, **`internet`**, **`romantic`**.

• Processed using **`OneHotEncoder(handle_unknown='ignore', sparse_output=False)`**.

• Preprocessing pipeline is fitted strictly on the training set only.

• Serialized artifacts saved in **`models/`**:

• **`models/preprocessor.joblib`**: fitted scikit-learn **`ColumnTransformer`**.

• **`models/feature_metadata.json`**: input column names, types, allowable ranges, and transformed feature names.

• **`data/reference/reference_features.parquet`**: training feature distribution for drift calculation.

3.3 Model Training & Comparison (`src/training/train.py`, `src/training/evaluate.py`)

• **Splits**: Stratified split — 70% Train, 15% Validation, 15% Test with **`random_state=42`**.

• **Candidate Models**:

1. **Logistic Regression (Baseline)**: with **`class_weight='balanced'`**.
2. **Random Forest Classifier**: **`n_estimators=100`**, **`max_depth=8`**, **`class_weight='balanced'`**.
3. **XGBoost Classifier**: **`n_estimators=100`**, **`learning_rate=0.05`**, **`scale_pos_weight=3.6`**.

• **Evaluation Protocol**:

• Metrics computed on holdout test set: Accuracy, Precision, Recall, F1-Score, ROC-AUC, PR-AUC, Confusion Matrix, and full Classification Report.

• Generates evaluation artifacts: **`reports/model_comparison.json`**, **`reports/confusion_matrix.png`**, **`reports/roc_curves.png`**, **`reports/feature_importance.png`**.

• **Champion Selection**: Model selected based on validation set F1-Score and Recall, saved as **`models/best_model.joblib`** and logged in **`models/model_registry.json`**.

3.4 MLflow Experiment Tracking (`src/tracking/mlflow_utils.py`)

• Configured with local tracking URI: **`sqlite:///mlruns.db`** (or **`./mlruns`**).

• Experiment: **`student_performance_mlops`**.

• Each candidate model logged in an isolated run with parameters, metrics, artifacts, and tags.

• Logs the serialized model artifact into MLflow.

3.5 Dataset and Model Versioning

• **`params.yaml`**: Centralized configuration for seed, split ratios, model hyperparameters, and drift thresholds.

• **`dvc.yaml`**: DVC pipeline stages (**`features`**, **`train`**, **`evaluate`**).

• **`data/reference/dataset_fingerprint.json`**: SHA-256 data hash, record count, column count, and class distribution metadata.

3.6 FastAPI Prediction Service (`api/`)

• **`api/schemas.py`**: Pydantic input/output schemas with field constraints (e.g. **`g1`** between 0 and 20, valid categorical domains).

• **`api/services.py`**: Model loading, cached in memory on startup, inference with probability scoring and risk category interpretation.

• **`api/main.py`**:

• **`GET /health`**: Service health and model load status.

• **`GET /model-info`**: Active model name, version, training date, target, test metrics.

• **`POST /predict`**: Single student prediction.

• **`POST /predict/batch`**: Multiple students prediction.

• **`GET /metrics`**: Prometheus-compatible / structured JSON metrics (request count, latency stats, error rate).

• Structured logging, graceful error handling (400 Bad Request, 503 Service Unavailable).

3.7 Docker Containerization

• **`Dockerfile`** / **`api/Dockerfile`**: Multi-stage or lightweight Python 3.12 image for the FastAPI service.

• **`.dockerignore`**: Excludes **`.git`**, **`__pycache__`**, large temporary files.

• **`docker-compose.yml`**: Multi-container architecture with **`postgres`**, **`fastapi_api`** (port 8000), and **`streamlit_dashboard`** (port 8501).

3.8 Streamlit Dashboard Extension (`dashboard/app.py`)

Preserves Views 1–5 from Part 1, adding:

- **View 6: ■ Student Risk Prediction (MLOps)**:
- Interactive student selector or manual student profile input form.
- Calls FastAPI **`/predict`** (with seamless local fallback if API server is not running).
- Displays risk badge (**■ `HIGH RISK OF FAILURE`** vs **■ `LOW RISK / ON TRACK`**), predicted risk probability gauge, key risk drivers, and pedagogical recommendations.
- **View 7: ■ MLOps Monitoring & Drift Diagnostics**:
- Live model version, training metrics, and latency statistics.
- Live data quality summary (missing rate, invalid values).
- Feature drift diagnostic table showing Population Stability Index (PSI) per feature and overall drift status (**■ `STABLE`**, **■ `MODERATE DRIFT`**, **■ `CRITICAL DRIFT`**).

3.9 Monitoring & Retraining Pipeline (`src/monitoring/`, `src/training/retrain.py`)

- **`src/monitoring/drift.py`**: Computes Population Stability Index (PSI) between incoming data and reference training set.
- **`src/monitoring/data_quality.py`**: Validates runtime inference input schemas and range integrity.
- **`src/training/retrain.py`**: Automated retraining workflow that trains a candidate model on new/updated data, evaluates against the holdout test set, and conditionally promotes the candidate only if F1-score/Recall outperforms the champion model by *\ge 1.0\%*.

3.10 Automated Testing Suite (`tests/`)

- **`tests/test_features.py`**: Tests preprocessor fitting, transformation, shape consistency, and lack of leakage.
- **`tests/test_training.py`**: Tests model training, hyperparameter execution, metric calculation, and artifact persistence.
- **`tests/test_api.py`**: FastAPI endpoint tests via **`httpx`** / **`TestClient`** (**`/health`**, **`/model-info`**, **`/predict`**, **`/predict/batch`**, invalid inputs).
- **`tests/test_monitoring.py`**: Tests data quality validation, PSI drift calculation, and retraining promotion logic.
- **`.github/workflows/ci.yml`**: GitHub Actions automated CI testing workflow.

---

4. Phase-by-Phase Implementation Roadmap

1. **Phase 1: Repository Inspection & Target Setup**
- Verify dataset integrity, establish **`data/reference/dataset_fingerprint.json`** and **`params.yaml`**.
2. **Phase 2: Feature Engineering & Preprocessing Pipeline**
- Build **`src/features/build_features.py`**, fit preprocessor, save **`models/preprocessor.joblib`** and metadata.
3. **Phase 3: Model Training, Evaluation & MLflow Experiment Tracking**
- Implement **`src/training/train.py`**, **`src/training/evaluate.py`**, **`src/tracking/mlflow_utils.py`**.
- Train Logistic Regression, Random Forest, XGBoost; log to MLflow; save champion model.
4. **Phase 4: FastAPI Prediction Service**
- Build **`api/schemas.py`**, **`api/services.py`**, **`api/main.py`**. Test endpoints with unit tests.
5. **Phase 5: Docker Containerization**
- Update **`Dockerfile`**, **`.dockerignore`**, **`docker-compose.yml`**.
6. **Phase 6: Streamlit Dashboard Integration**
- Add View 6 (Risk Prediction) and View 7 (MLOps Monitoring) to **`dashboard/app.py`**.
7. **Phase 7: Monitoring & Retraining Pipeline**
- Implement **`src/monitoring/drift.py`**, **`src/monitoring/data_quality.py`**, **`src/training/retrain.py`**.
8. **Phase 8: Comprehensive Testing, Documentation & CI**
- Complete **`tests/`**, update **`README.md`**, add **`.github/workflows/ci.yml`**, verify end-to-end.

---

5. Verification Plan

Automated Tests

- Run **`python -m pytest tests/ -v`** to verify:

1. Part 1 data engineering tests (13 tests) remain 100% passing.
2. Part 2 feature engineering, model training, FastAPI endpoints, drift detection, and retraining tests all pass.

Execution Verification

- Train models: **`python src/training/train.py`**
- Run API: verify **`http://localhost:8000/health`**, **`/model-info`**, **`/predict`**, **`/metrics`**
- Run Dashboard: verify all 7 views render smoothly on **`http://localhost:8501`**
- Test Retraining: **`python src/training/retrain.py`**
- Test Drift Monitoring: **`python -c "from src.monitoring.drift import run_drift_analysis; print(run_drift_analysis())"`**